

Correct Optimizations: Phase 2 Report Based on “Minotaur: A SIMD-Oriented Synthesizing Superoptimizer” [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

1 Introduction

Minotaur [1] is a superoptimizer, that after computing optimizations, caches those in a data base. For this purpose it uses the original code as the key to find a rewrite/optimization which is the value. The intention behind the caching is to speedup the substantial runtimes of Minotaur for future compilations of the same or similar programs. Minotaurs compile times, with a cold cache, can easily be hundreds of times slower than with regular LLVM [1].

Currently Minotaur directly uses the extracted LLVM IR code segments as the keys and stores those along side a rewrite, in Minotaurs own IR, as values in the cache. An example for a key-value pair is shown in figure 1.

This approach, of directly using the extracted code segments, can easily lead to near misses, where an equivalent code segment is already cached, but for instance has unused arguments, leading to a cache miss and a costly recompute of an optimization.

To avoid this issue we attempt to increase the number of cache hits. This leads to reduced compile times of Minotaur even when starting from a cold cache. The main challenge is to maintain correctness of the cached optimizations.

2 Research Proposal

To address the issue of near cache misses, our solution introduces an additional preprocessing step before a key is used for a cache lookup. It performs multiple code transformations to map semantically equivalent code to a "canonical" representation.

The idea is that this leads to fewer cache entries, an increased cache hit rate and fewer slow optimization searches by Minotaur. For large enough projects this can already take effect during an initial build, even when starting with an empty cache and thereby reducing the compile time.

3 Canonicalization

To motivate the idea of canonicalization we start by defining the set of all valid LLVM IR snippets and call it the key space. The key space consists of equivalence classes of semantically equivalent keys. We call a set that contains one representative of each equivalence class a canonical key space. An ideal transformation would map each input LLVM IR snippet to its representative in a canonical key space. Using such a transformation would enable reaching the optimal cache hit rate and avoid unnecessary expensive optimization searches by Minotaur, since there would be no cache misses anymore where an equivalent snippet has already been processed.

Finding such a transformation is hard, therefore we attempted to only shrink the size of each equivalence class. For this relaxation to work, each equivalence class has to be finite. This is because the probability that two randomly selected keys from the same equivalence class are identical depends on the size of that class. Because Minotaur only generates slices up to a fixed depth, it is possible to achieve finite equivalence classes with a transformation normalizing the names in a code snippet.

Our extension is placed as a pre- and postprocessing step around Minotaurs inference step. We intercept the LLVM IR produced by the slicer and perform transformations on it to produce a modified version of the input, which is then forwarded to the inference step. In the following this set of transformations is called canonicalization. After the inference step is done and returns a rewrite in Minotaurs own IR, another set of transformations on the rewrite, is performed. This process is called de-canonicalization from now on. Figure 2 gives an overview of how our extension integrates into Minotaur.

These transformation steps are always defined as a pair which we call a canonicalization-step. Each canonicalization-step in our implementation consists of a canonicalization function and a de-canonicalization function. The canonicalization-steps are ordered as to not interfere with each other. More on that is discussed in section 3.2. During the canonicalization process every canonicalization function is applied iter-

Figure 1: Example of a key value pair as it is found in the Minotaur cache.

```
key:
    define i1 @cut(i64 %__n2, i16 %__n3) {
        %__n0 = sub nsw i64 0, %__n2
        %__n1 = icmp samesign ult i64 %__n0, 2033
        ret i1 %__n1
    }

value:
    (icmp_sgt (var i64 %__n2) (reservedconst i64 |i64 -2033|) b1)
```

actively in this predefined order. For de-canonicalization the de-canonicalization functions are applied in the reverse order in which the canonicalization-steps are specified. It is also possible to pass information between the pairs of canonicalization and de-canonicalization functions of each step if required. This enables the support for more complex transformations.

Due to the implementation of Minotaur and how we integrated our extension all the de-canonicalization could be done by updating meta data already required by minotaur during the canonicalization instead of relying on the explicit de-canonicalization.

Our modified Minotaur version is available on GitHub ¹.

3.1 Canonicalization Steps

In the following each canonicalization-step is described. They were selected such that each step maintains semantical equivalence, reduces the size of the key space and that it does not modify the code snippets depth. Here two code snippets are considered equivalent if they compute the same value and have the same effects on memory, including ordering of changes during execution, when using the same arguments. These three properties are shown for each step.

It might be possible to relax the correctness property, to, for instance, allow the reordering of memory operations depending on the memory model. Similarly, the requirement of reducing the size of the key space could be relaxed such that it only holds over all steps but not for every individual step. Also, the depth constraint is not strictly required and could be dropped, but this would likely result in more complex solver queries which could drastically impact the overall runtime of Minotaur and offset any benefit gained from using canonicalization. This link between depth and runtime was already explored in the original paper [1].

An example of all the steps applied sequentially to an example input is shown in figure 3

Eliminating Unused Function Arguments Some of the functions extracted by Minotaur have unused arguments which can be removed during canonicalization. By removing

those the number of functions with the same body but different signatures are reduced. Thereby reducing the size of the key space. Assuming the given input function f has an unused argument we can define the following. Let $f : T_1 \times \dots \times T_i \times \dots \times T_n \rightarrow R$ and $g : T_1 \times \dots \times T_{i-1} \times T_{i+1} \times \dots \times T_n \rightarrow R$ be n -ary and $n - 1$ -ary functions respectively with

$$f(t_1, \dots, t_n) := g(t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_n), t_j \in T_j$$

Since t_i is not part of the function signature of g we can define a new function $f' := g$. This obviously does not change what is computed since

$$\begin{aligned} f(t_1, \dots, t_n) &= g(t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_n) \\ &= f'(t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_n) \end{aligned}$$

As shown in figure 1 the rewrite does not have an explicit function signature. Therefore no de-canonicalization is required. Modifying the function signature has no impact on the depth of the given code snippet.

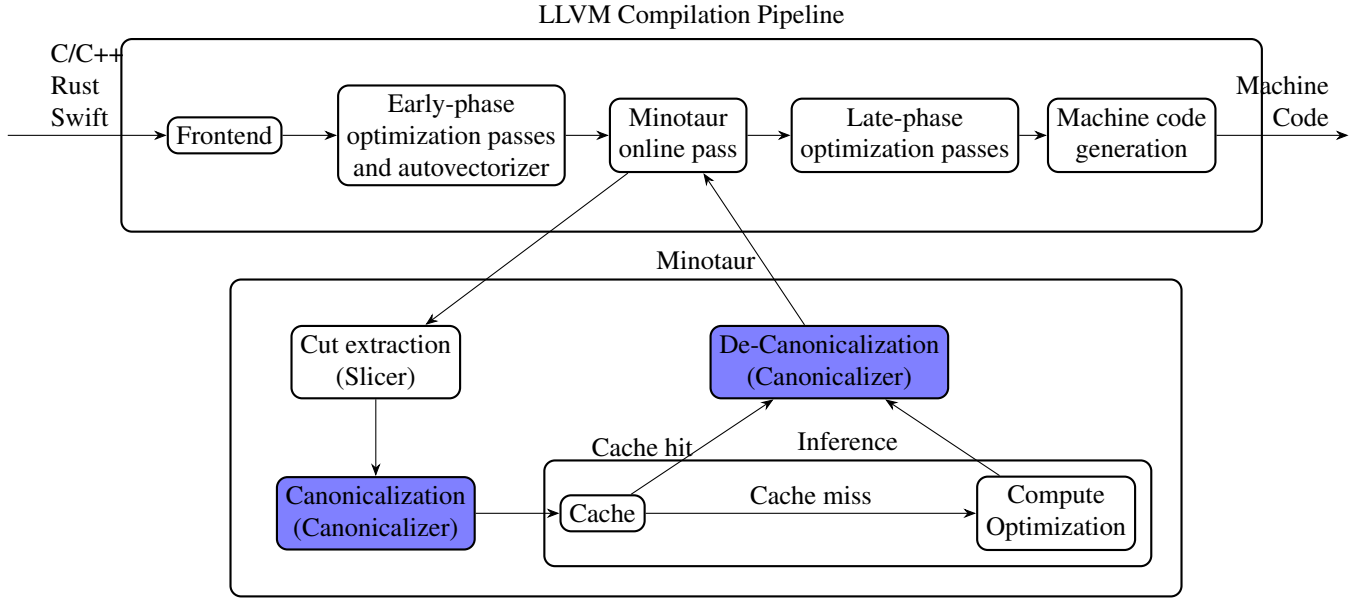
Sorting of Function Arguments Another modification that is done to the function signature is that the function arguments are sorted. The arguments are sorted by their type-name and internally each set of arguments of the same type is sorted by when they are used inside the function body. This defines a total order of function arguments for each function body and reducing the size of the key space. To show the correctness of this transformation we define $f : T_1 \times \dots \times T_n \rightarrow R$ as the input function and $f' : T_{\rho(1)} \times \dots \times T_{\rho(n)} \rightarrow R$ the output function with $\rho : [n] \rightarrow [n]$ being a bijective function i.e. a permutation. We define

$$\begin{aligned} f'(t_{\rho(1)}, \dots, t_{\rho(n)}) &:= f(t_{\rho^{-1}(\rho(1))}, \dots, t_{\rho^{-1}(\rho(n))}) \\ &= f(t_1, \dots, t_n), t_i \in T_i \end{aligned}$$

This clearly shows that f and f' compute the same for all inputs. Because Minotaur maintains a mapping between the extracted functions arguments and the original code the snippet was extracted from, we can simply update this mapping and do not require explicit de-canonicalization. Since only the function signature is modified there is no change to the depth.

¹Extended Minotaur Version

Figure 2: Structure of Minotaur and how the Canonicalization is integrated. The highlighted blocks were added for (De-)Canonicalization.



Renaming of Function Arguments After the previous two transformations the signature is almost unique for each function body with the exception of the naming. To resolve this each argument is renamed depending on its position in the signature. Because now the names of all function arguments are defined by their position all functions with a given signature and body map to the same key. Given a function $f(t_1, \dots, t_n) := g(t_1, \dots, t_n)$ as the input to the transformation step and a function $f'(x_1, \dots, x_n) := g(x_1, \dots, x_n)$ as the output. If we apply f and f' to the same set of arguments $v_i, i \in [n]$ we get

$$f(v_1, \dots, v_n) = g(v_1, \dots, v_n) = f'(v_1, \dots, v_n)$$

This shows that renaming the arguments does not change the result of the function. As mentioned previously, Minotaur maintains a mapping between the arguments of the extracted function and the original code snippet from which it was derived. This mapping does not rely on names but instead uses references to objects that possess names, eliminating the need for de-canonicalization. Renaming does not impact the depth of the function.

Replacing Greater-Than with Less-Than Comparison operations often have multiple semantically equivalent ways of being expressed. To address this every greater-than or greater-equal comparison is replaced with a less-than or less-equal comparison respectively and the operands are swapped. This reduces the size of the key space since now only less-than and less-equal comparisons are possible. The correctness

of the transformation follows from the "lemma" below.

$$\forall x \forall y. (x P y \iff y P' x) \\ \text{with } (P, P') \in (>, <), (\geq, \leq)$$

For this transformation no de-canonicalization is needed. The depth is not impacted, since only the predicate of a comparison operator is modified.

Non-strict Operator Replacement This transformation replaces non-strict comparison operators with strict ones whenever possible, subject to certain constraints. The replacement is applied only when the comparison involves constant operands, ensuring that the depth remains unchanged and that no additional operations are introduced. By reducing the number of distinct ways to express equivalent comparisons, this transformation shrinks the key space. Each value of LLVM's build-in integral and floating-point types represents a specific real number, with the exception of the special values *NaN* and $+\infty$. The real numbers form a total order; consequently, the set of all non-special values of each type also forms a total order. For floating-point types, this order can be extended by defining $-\infty$ as the minimum element and $+\infty$ as the maximum element. Within a total order each value, with the exception of the minimum and maximum, have a greater and smaller neighbor.

In a total order, every value—except for the minimum and maximum—has both an immediate predecessor and an immediate successor. The minimum has only a successor, and

the maximum has only a predecessor. Due to this property, non-strict comparison operators can be replaced by strict ones by appropriately adjusting a constant operand, provided that a suitable neighboring value exists.

For example, given an expression of the form $x \leq c$ where x is a variable and c is a constant, the expression can be transformed into $x < c + \epsilon$ where ϵ is the smallest possible increment for c . Formally:

$$\exists v \exists \epsilon \forall x. ((v = c + \epsilon) \implies (x \leq c \iff x < c + \epsilon))$$

Similar arguments can be made for the $\geq$ and for cases where the constant appears on the left-hand side of the comparison. No de-canonicalization is required for this transformation.

3.2 Ordering of the Canonicalization Steps

It is important to order the canonicalization-steps such that they do not interfere with previous ones. In our case this is still relatively simple since we only have two constraints.

First, the argument renaming step should be run after the last modification to functions arguments. In our case this implies that it has to run after the elimination of unused arguments step and after the argument sorting step.

Second, the sorting of function arguments has to take place after the last modification to the function body in which the usage order of the arguments is changed. This means concretely that it has to be run after the step replacing greater than with less than operators.

Based on those two constraints we can define the following order:

1. Elimination of unused Function Arguments
2. Replacing Greater-Than with Less-Than
3. Non-strict Operator Replacement
4. Sorting of Function Arguments
5. Renaming of Function Arguments

4 Evaluation

4.1 Setup

For the evaluation we used an AMD Ryzen 9 5900 CPU which has 12 cores running at up to 4.8 GHz. All benchmarks were build using the `-O3` and `-march=native` flags.

Each benchmarked program was build once using Minotaur with canonicalization disabled and once with canonicalization enabled. Since our canonicalization steps are compositional we measured the effect each step and combination of steps have. Furthermore, we ran the provided tests and benchmarked each compiled program.

We chose to benchmark libgmp-6.2.1 as the original paper did. Due to the long compile times experienced on libYUV, we decided against benchmarking it, like the original paper did, and instead opted to select different smaller projects.

4.2 Results

5 Discussion

6 Conclusion

To further improve our approach, one could attempt to perform a more structured search for additional transformations by finding syntactically equivalent rewrites in the cache and looking at their respective keys to come up with new transformations.

References

- [1] Zhengyang Liu, Stefan Mada, and John Regehr. “Minotaur: A SIMD-Oriented Synthesizing Superoptimizer”. In: *Proceedings of the ACM on Programming Languages (OOPSLA2)* 8.OOPSLA2 (Oct. 2024). <https://doi.org/10.1145/3689766>, pp. 1–25. DOI: 10.1145/3689766.

Figure 3: Example applying each canonicalization step.

input:

```
define i1 @cut(i32 %a, float %b, i32 %c, float %d) {
    %1 = fcmp uge float %b, %d
    %2 = icmp sle i32 %c, 15
    %3 = and i1 %1, %2
    ret i1 %3
}
```

unused argument elimination:

```
define i1 @cut (float %b, i32 %c, float %d) {
    %1 = fcmp uge float %b, %d
    %2 = icmp sle i32 %c, 15
    %3 = and i1 %1, %2
    ret i1 %3
}
```

replacing greater-than with less-than:

```
define i1 @cut(float %b, i32 %c, float %d) {
    %1 = fcmp ule float %d, %b
    %2 = icmp sle i32 %c, 15
    %3 = and i1 %1, %2
    ret i1 %3
}
```

replacing non-strict operators:

```
define i1 @cut(float %b, i32 %c, float %d) {
    %1 = fcmp ule float %d, %b
    %2 = icmp slt i32 %c, 16
    %3 = and i1 %1, %2
    ret i1 %3
}
```

sorting arguments:

```
define i1 @cut (float %d, float %b, i32 %c) {
    %1 = fcmp ule float %d, %b
    %2 = icmp slt i32 %c, 16
    %3 = and i1 %1, %2
    ret i1 %3
}
```

renaming argument:

```
define i1 @cut (float %__canonArg, float %__canonArg1, i32 %__canonArg2) {
    %1 = fcmp ule float %__canonArg, %__canonArg1
    %2 = icmp slt i32 %__canonArg2, 16
    %3 = and i1 %1, %2
    ret i1 %3
}
```